

A3-PS022-TC367

eBPF/XDP Rate Limiting for DNS Query Floods in a Lab

Keerthi Virupapura
keerthivirupapura.cs25@bmsce.ac.in
U Kruthik
ukruthik.cs25@bmsce.ac.in
Dr. Manjunath D R (Faculty)
manjunathdr.cse@bmsce.ac.in

AIORI Hackathon 3
October 11, 2026

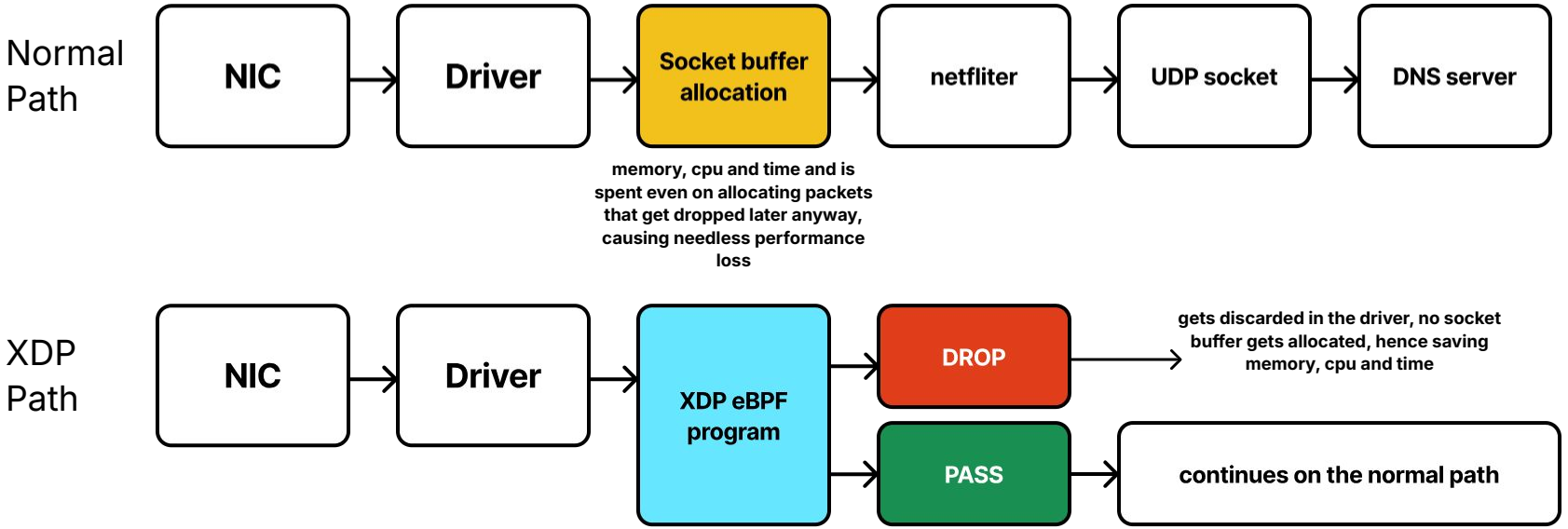
Problem Description

Cheap to send, costly to receive: high-volume UDP DNS query floods use small, easily generated packets to saturate a Linux server.
The kernel pays first: every packet triggers sk_buff allocation and network-stack processing before the DNS server even sees it, whether or not it will be discarded.
Legitimate users suffer: CPU saturation and full socket buffers cause dropped and delayed answers for real queries.
Late filtering gives limited relief: firewall rules such as nftables run after the sk_buff exists, even at the earliest hook, so much of the cost is already paid.
Rate limiting is a trade-off: per-source limits miss floods from many sources, and global limits can drop legitimate queries too.
No fair evidence yet: we need a repeatable comparison of no filter vs nftables vs XDP on CPU, drops and legitimate-query success

Our Challenge: Build a bounded-parsing XDP filter with per-source and global rate limits, and measure its effect on legitimate queries in a controlled lab.

Background

- **XDP moves the drop decision ahead of the kernel's per-packet cost**



Literature & RFC Review

Source	What it says	Key Takeaway
RFC 1035: Domain names, implementation and specification	Defines the DNS message layout. including the fixed 12 byte header	basis of bounded parsing, we read only this header and never the query name
RFC 2827 / BCP 38: Network Ingress Filtering	Asks networks to drop packets that carry forged source addresses	Spoofed sources defeat per source limits, so a global limit is also needed
DNS response rate limiting (ISC RRL; in BIND, NSD, Knot)	Limits repeated identical responses at the DNS server, in user space	Complementary: it acts on responses after the packet cost is paid, we act on queries in the driver
Hoiland Jorgensen et al: the eXPD, coNEXT 2018	Presents XDP's design: shows driver-level processing reaches far higher per core packet rates than the normal stack	Motivates our hypothesis that XDP spends less CPU per dropped packet than nftables, which we test on DNS floods instead of assuming it
Public XDP based DDoS mitigation wrte-ups	Describes dropping attack traffic early with XDP in production	Shows the approach is used in practice, we add a bounded DNS specific lab measured comparison
GCRA Algorithm	Rate control that stores a single timestamp per flow	Basis of our integer only limiter

Proposed Solution

XDP Inspection & Filtering: Attach a small XDP program ahead of the kernel receive path to inspect fixed-size headers (Ethernet, IPv4, UDP, and 12-byte DNS header) without reading query names, passing non-DNS traffic untouched and dropping malformed DNS packets immediately.

Rate Limiting & Memory Protection: Enforce ordered per-source and global GCRA token buckets using integer math and a bounded LRU map—dropping excess packets directly in the driver before kernel buffer allocation to prevent memory exhaustion from spoofed floods.

Observability & Runtime Control: Count every verdict by reason and allow limit parameters to be updated dynamically at runtime without reloading the XDP program.

Benchmark Testbed: Evaluate the system on an isolated pair of Linux hosts by running identical traffic across four configurations: no filter, two nftables rule sets (prerouting and netdev ingress), and XDP.

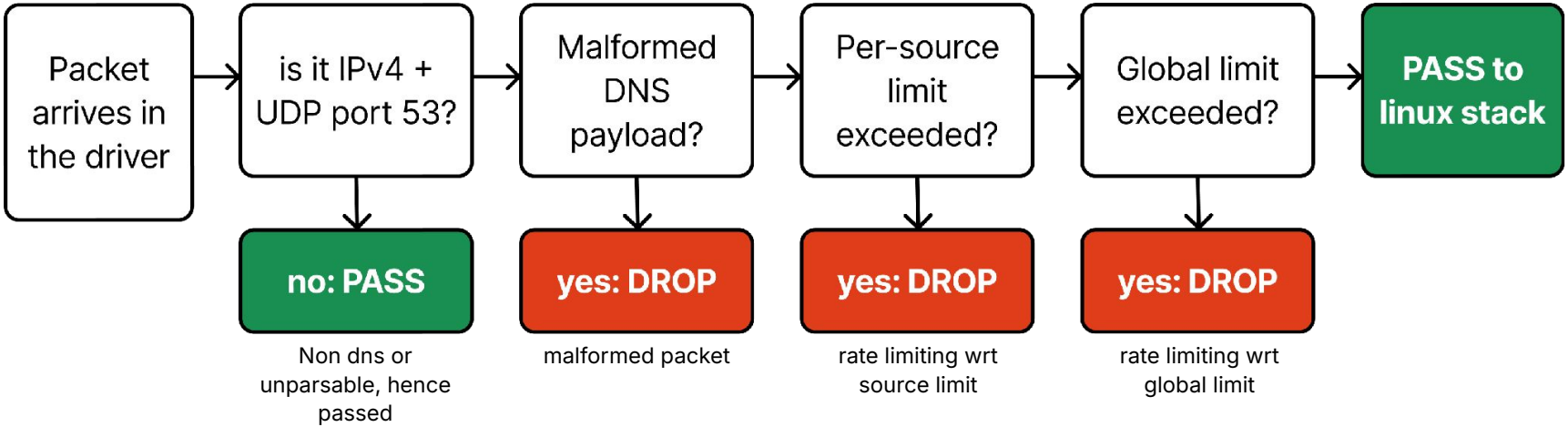
Flood Testing & Metrics: Measure system response using a legitimate client sending queries during three synthetic floods (single source, many sources, and pulsing bursts) to compare CPU usage, packet drops, and answered legitimate queries.

3
Flood Scenarios

4
Test Configurations

240
planned runs, 5 reps per cell

Proposed Solution



Optimization Proposed

Aspect	Before (typical baseline	After (our approach)	What we report
Drop point	After sk_buff allocation	In the driver, before allocation (native XDP)	CPU per dropped packet
Parsing	Full stak processing, then the DNS server	Bounded Ethernet, IPv4, UDP and 12-byte DNS header; malformed packets dropped early	Verifier acceptance and malformed count
Per-source state	nftables dynamic set, no eviction and can get filled up	LRU hashmap, bounded memory and the oldest entries are evicted	Behaviour under many sources
Limiter math	Token refill arithmetic with division	GCRA: one u64 per source, integers only	Limiter accuracy (passed vs configured rate)
Check order	Global limit first, which can starve legitimate users	per-source first and then global	Legitimate success under a single source flood
Tuning	Rebuild or reload to change limits	Runtime config map, we can sweep rates without reload	Scenario sweeps
Evaluation	Ad hoc flood rates	Calibrated levels (0.5x to 5x saturation), any order, drops by layer	Median and spread of 5 runs

Solution Architecture & Design

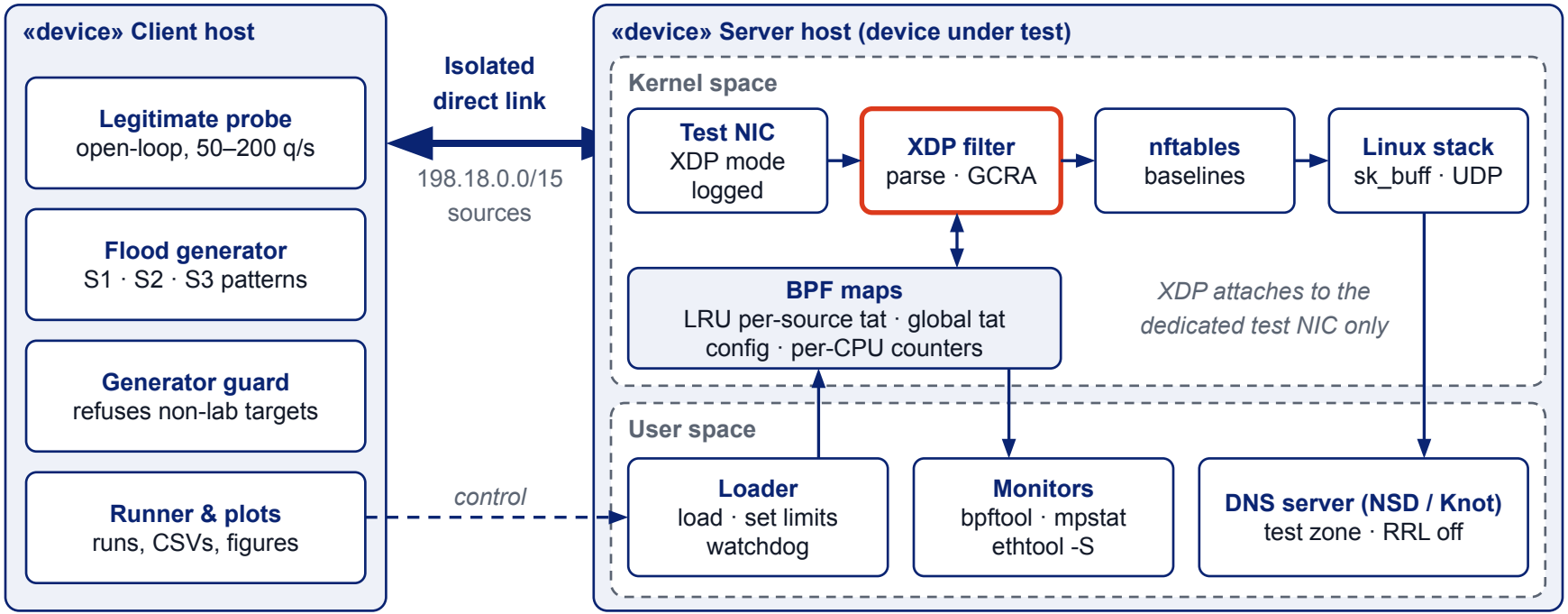
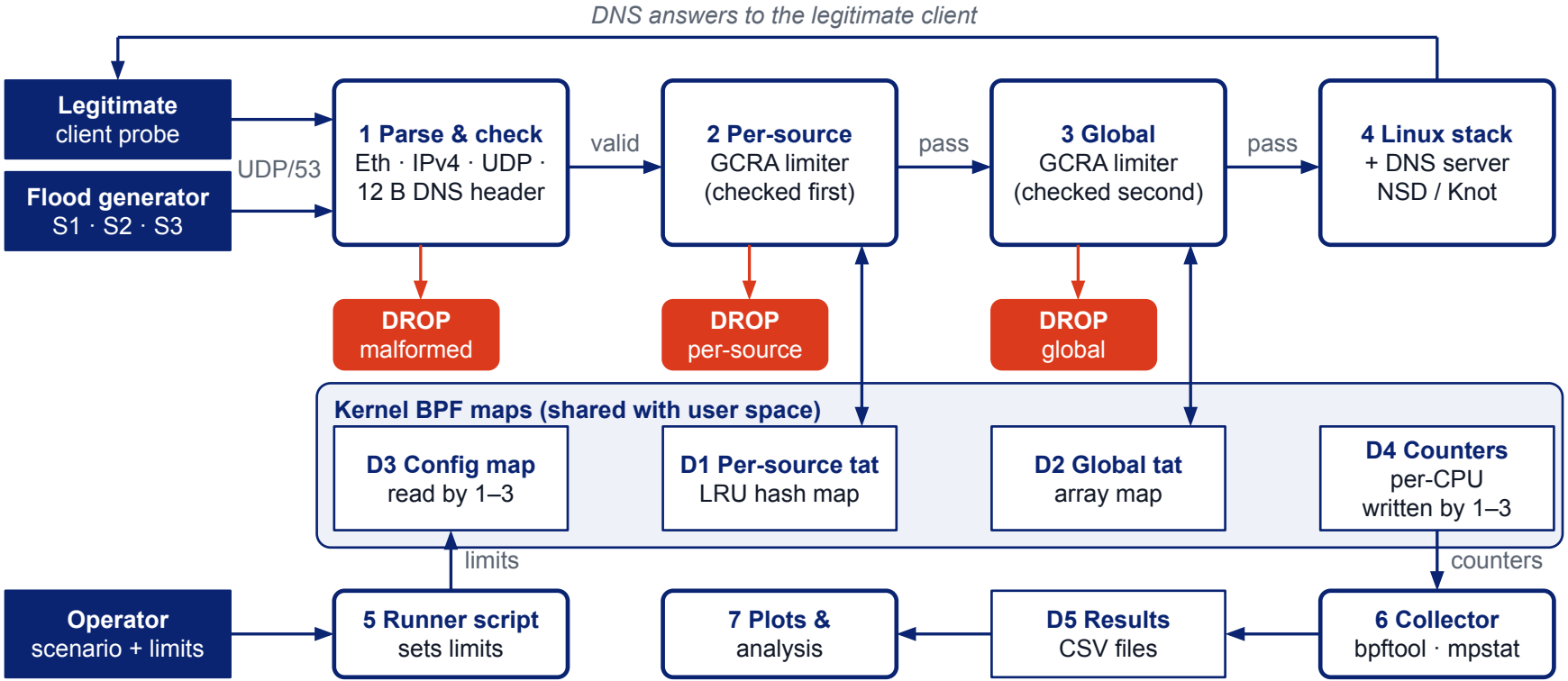
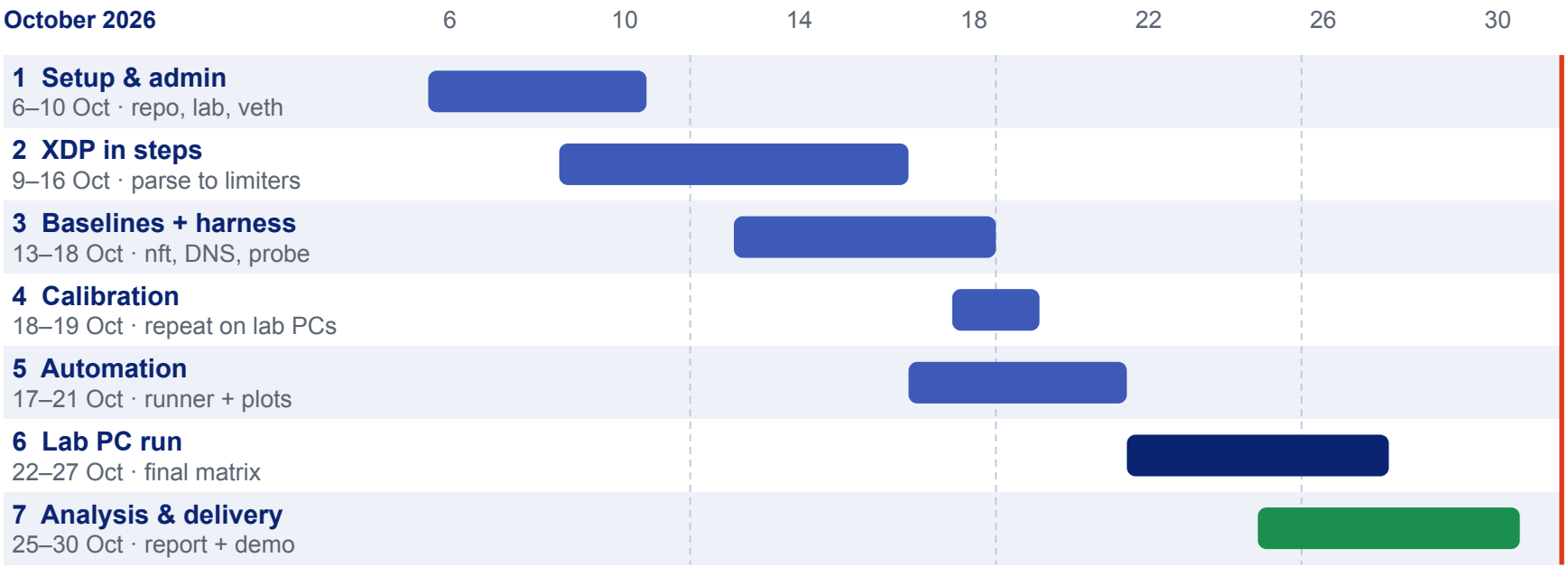


Figure: System architecture (UML component / deployment view). Solid arrows: packet and data path; dashed: control.

Data Flow Diagram



Deployment Timeline



Method: validate in namespaces + veth first, then repeat on isolated lab PCs. Matrix: 4 configs × 3 scenarios × 4 flood levels × 5 reps = 240 runs (10 s warm-up + 60 s measurement), randomised order, state reset between runs.

Bibliography

1. P. Mockapetris, “Domain Names – Implementation and Specification,” RFC 1035, IETF, Nov. 1987.

2. P. Ferguson and D. Senie, “Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing,” BCP 38 (RFC 2827), IETF, May 2000.

3. P. Vixie and V. Schryver, “DNS Response Rate Limiting (DNS RRL),” ISC Technical Note ISC-TN-2012-1-Draft1, Apr. 2012.

4. T. Høiland-Jørgensen et al., “The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel,” in Proc. ACM CoNEXT, Heraklion, Greece, Dec. 2018, pp. 54–66.

5. A. Fabre, “L4Drop: XDP DDoS Mitigations,” Cloudflare Blog, 2018. blog.cloudflare.com/l4drop-xdp-ebpf-based-ddos-mitigations

6. ITU-T, “Traffic control and congestion control in B-ISDN,” Recommendation I.371 (Annex A: Generic Cell Rate Algorithm), 2004.



Thank You for Your Attention!

Questions?

Keerthi Virupapura
keerthivirupapura.cs25@bmsce.ac.in

A3-PS022-TC367 · eBPF/XDP Rate Limiting for DNS Query Floods in a Lab
AIORI Hackathon 3